

题目链接

UVa11255

题意

有三种不同颜色的珠子，各有a、b、c个， $3 \leq a + b + c \leq 40$ （a、b、c非负），求将所有珠子连成一串项链能组成的不同方案数。旋转或翻转后相同的方案认为是同种方案。

分析

本题是[项链和手镯](#)的一个变体，这里是固定三种颜色且每种颜色的珠子的数量也固定的情况下求方案数。

这里要说一下[Burnside 引理](#)，对于置换f求不动点计数C(f)的一点深入理解。对于某个置换f，只要其存在（本题指的是可以旋转或者翻转），则不管颜色限制是什么，它给分母贡献加1，只是特定的颜色限制下C(f)可能为0（即此置换在特定颜色限制下循环无法全部得到满足，对分子贡献加0）。计 $n = a + b + c$ ，本题分母总是2n, (不旋转，n-1种旋转，n种翻转）。

本题的所有旋转与翻转的置换方案都明显存在（因此不需要判断存在性），而[苏丹的吊灯](#)这个题目，则需要判断某个置换f是否存在，因为循环对应的的每个子吊灯需要形状相同才允许旋转。

AC代码

```
#include <iostream>
#include <numeric>
using namespace std;

#define N 41
long long c[N][N], s; int a, b, d, n;

void rot(int r) {
    int g = gcd(n, r), l = n/g;
    if (a%l || b%l || d%l) return;
    int x = a/l, y = b/l, z = d/l;
    s += c[x+y+z][x] * c[y+z][y];
}

void flip() {
    int f = (a&1) + (b&1) + (d&1);
    if ((n&1 && f!=1) || (~n&1 && f&1)) return;
    int x = a>>1, y = b>>1, z = d>>1;
    if (f) s += n * c[x+y+z][x] * c[y+z][y];
    else {
        s += n/2 * c[x+y+z][x] * c[y+z][y];
        if (a > 0) s += n/2 * c[x+y+z-1][x-1] * c[y+z][y];
        if (b > 0) s += n/2 * c[x+y+z-1][y-1] * c[z+x][z];
        if (d > 0) s += n/2 * c[x+y+z-1][z-1] * c[y+x][y];
    }
}

void solve() {
    cin >> a >> b >> d;
    n = a + b + d; s = c[n][a] * c[b+d][b];
    for (int i=1; i<n; ++i) rot(i);
    flip();
    cout << s/2/n << endl;
}

int main() {
    c[0][0] = 1;
    for (int i=1; i<N; ++i) {
        c[i][0] = 1;
        for (int j=1; j<=i; ++j) c[i][j] = c[i-1][j-1] + c[i-1][j];
    }
    int t; cin >> t;
    while (t--) solve();
    return 0;
}
```